

reduces payload sizes while maintaining API versatility.

Pagination prevents massive responses by limiting results per page. Django REST Framework provides built-in pagination classes. Configure a default page size and maximum page size to balance performance and usability.

Enabling GZIP compression

GZIP compression reduces payload sizes by 60-80% for text-based responses like JSON, HTML, CSS, and JavaScript. The bandwidth savings are substantial, especially for mobile users on slow connections.

Django's `GZipMiddleware` automatically compresses responses larger than 200 bytes. Add it early in the middleware list for maximum coverage. However, NGINX compression is preferred for production because it offloads compression work from Django.

Configure NGINX's gzip settings to enable compression, set compression level, specify MIME types to compress, and set minimum length. NGINX handles compression efficiently, freeing Django to focus on application logic.

```
#settings.py - Django approach
MIDDLEWARE = [
    'django.middleware.gzip.GZipMiddleware',
    #... other middleware
]
```

Load balancing with NGINX

NGINX distributes requests across multiple Django instances, improving throughput and providing redundancy. Configure an upstream block defining backend servers, then proxy requests to this upstream.

NGINX supports multiple load balancing algorithms. 'Least connections' routes requests to the server with the fewest active connections. 'Round robin' distributes evenly. 'IP hash' ensures the same client always reaches the same backend.

Run multiple Gunicorn instances on different ports, each with multiple workers. NGINX distributes incoming requests across these instances. If one instance fails, NGINX automatically routes traffic to healthy instances.

Implementing API gateways

API gateways provide centralised entry points for microservices, handling authentication, rate limiting, request routing, and monitoring. Kong is an open source API gateway built on NGINX and OpenResty.

Kong manages services and routes. Define each microservice in Kong, create routes for URL patterns, and enable plugins for features like rate limiting, authentication, and logging. Kong handles these concerns centrally, simplifying individual microservice implementations.

This centralisation reduces code duplication. Instead of implementing rate limiting in every microservice, configure it once in Kong. Authentication, logging, and monitoring work similarly.

```
#Add service in Kong
curl -X POST http://localhost:8001/services/
  --data "name=product-service"
  --data "url=http://127.0.0.1:8002"

#Add route
curl -X POST http://localhost:8001/services/product-
service/routes
  --data "paths[]=products"

#Enable rate limiting
curl -X POST http://localhost:8001/services/product-
service/plugins
  --data "name=rate-limiting"
  --data "config.minute=100"
```

Monitoring and observability

Performance optimisation requires continuous monitoring to identify bottlenecks and validate improvements. Prometheus collects metrics, Grafana visualises them, and Sentry tracks errors and performance issues.

Django-prometheus integrates Prometheus metrics collection into Django. It tracks request counts, response times, database query times, cache hit rates, and more. Prometheus scrapes these metrics periodically, storing them for analysis.

Grafana creates dashboards from Prometheus data. Visualise response time distributions, identify slow endpoints, track database query performance, and monitor cache effectiveness. These insights guide optimisation efforts.

Sentry provides error tracking and performance monitoring. It captures exceptions with full stack traces, tracks slow database queries, identifies N+1 query problems, and monitors transaction performance. This visibility is essential for production applications.

```
#Install Sentry SDK
pip install sentry-sdk

#settings.py
import sentry_sdk
from sentry_sdk.integrations.django import
DjangoIntegration
sentry_sdk.init(
    dsn="https://your-dsn@sentry.io/project",
    integrations=[DjangoIntegration()],
```